

Lab 03

Bruce Trevisan

Dados relacionais e operações do tipo join

Atrasos de vôos

1. Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

```
library(readr)
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

filter, lag

Os seguintes objetos são mascarados por 'package:base':

intersect, setdiff, setequal, union

```
library(stringr)
library(tidyr)
```

```
airlines <- read_csv("archive/airlines.csv")
```

Rows: 14 Columns: 2

-- Column specification -----

Delimiter: ","

chr (2): IATA_CODE, AIRLINE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

- Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;

```
airports <- read_csv(  
  "archive/airports.csv",  
  col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)  
)
```

Rows: 322 Columns: 5

-- Column specification -----

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

- Para o arquivo de vôos:
 - a. Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY;
 - b. Leia apenas 1 milhão de linhas por vez;

```
flights <- read_csv_chunked(  
  file = "archive/flights.csv",  
  callback = DataFrameCallback$new(function(chunk, pos) chunk),  
  chunk_size = 100000, # 1.000.000 não foi, por isso usei 100.000  
  col_types = cols_only(  
    DESTINATION_AIRPORT = col_character(),  
    ARRIVAL_DELAY = col_double()  
  )  
)
```

- c. Remova vôos em que o aeroporto de destino comece com a letra 1;
- d. Remova registros em que existam pelo menos uma coluna faltante;

```

flights <- flights %>%
  filter(!str_starts(DESTINATION_AIRPORT, "1"))

flights <- flights %>%
  drop_na()

```

- e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;

As estatísticas suficientes para a determinação do atraso médio vão ser a soma total dos atrasos e a quantidade total de voos de cada aeroporto.

- f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```

estatisticas <- read_csv_chunked(
  file = "archive/flights.csv",
  chunk_size = 500000,
  col_types = cols_only(DESTINATION_AIRPORT = "c", ARRIVAL_DELAY = "d"),
  callback = DataFrameCallback$new(function(chunk, pos) {
    chunk %>%
      filter(!stringr::str_starts(DESTINATION_AIRPORT, "1")) %>%
      na.omit() %>%
      group_by(DESTINATION_AIRPORT) %>%
      summarise(
        soma_atraso = sum(ARRIVAL_DELAY),
        qtd_voos = n(),
        .groups = "drop"
      )
  })
)

atrasos_final <- estatisticas %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    atraso_medio = sum(soma_atraso) / sum(qtd_voos),
    .groups = "drop"
  )

```

2. Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa:

- a. Identifique a coluna que é a chave na tabela flights;

A chave na tabela flights é a coluna DESTINATION_AIRPORT, pois ela corresponde à coluna IATA_CODE da tabela airports.

- b. Identifique a coluna que é a chave na tabela airports;

A chave na tabela airports é a coluna IATA_CODE.

- c. Quais são os aeroportos que estão listados em flights, mas d. estão ausentes em airports?

```
aeroportos_ausentes <- flights %>%  
  select(DESTINATION_AIRPORT) %>%  
  distinct() %>%  
  anti_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))  
  
print(aeroportos_ausentes)
```

```
# A tibble: 0 x 1  
# i 1 variable: DESTINATION_AIRPORT <chr>
```

- d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
e. Armazene a tabela resultante no objeto flights.

```
flights <- flights %>%  
  left_join(  
    airports %>% select(IATA_CODE, CITY, STATE),  
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")  
  )
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
aeroportos_por_estado <- airports %>%  
  count(STATE, name = "total_aeroportos") %>%  
  arrange(desc(total_aeroportos))  
  
print(aeroportos_por_estado)
```

```
# A tibble: 54 x 2
  STATE total_aeroportos
  <chr>         <int>
1 TX             24
2 CA             22
3 AK             19
4 FL             17
5 MI             15
6 NY             14
7 CO             10
8 MN              8
9 MT              8
10 NC             8
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

- a. Carregue o pacote leaflet;

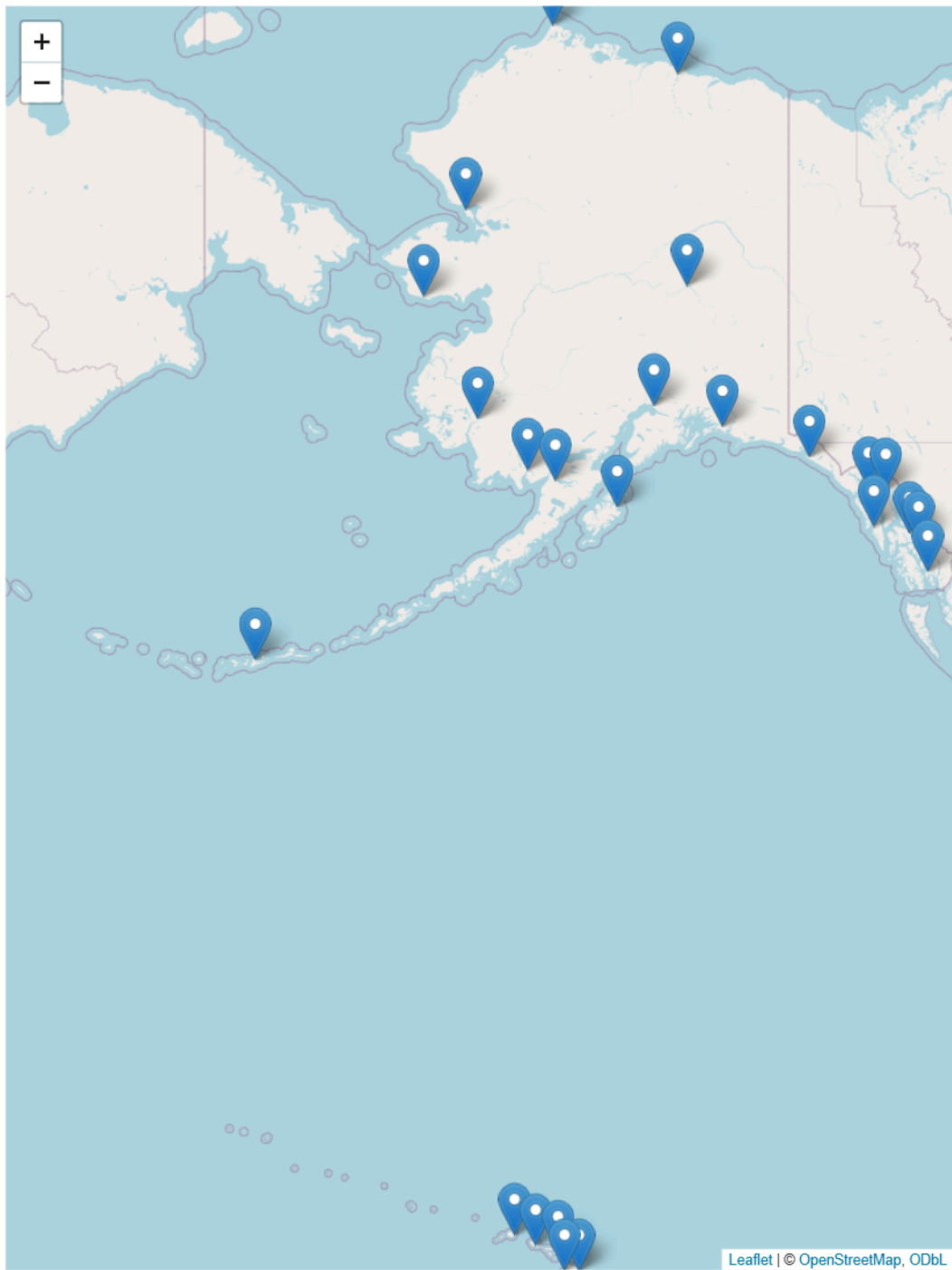
```
library(leaflet)
```

- b. Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico;
- c. Armazene o grafico em b) numa variável chamada this;

```
this <- leaflet(airports) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~CITY
  )
```

Warning in validateCoords(lng, lat, funcName): Data contains 3 rows with either missing or invalid lat/lon values and will be ignored

```
this
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 22:29:44 -03"
```